

Assignment No: 8

Student Details

Name: Tanushka Patil

PRN / Roll No: 123B1B036

Class: BTech

Division: A

Subject: DevOps

Title

Explore Kubernetes Objects and Services and Demonstrate Configuration Management Using Ansible

Aim

To study important Kubernetes objects and networking services and demonstrate the use of Ansible for automated configuration management.

Theory

Part A: Kubernetes Objects

- **Pod:** the smallest deployable unit; one or more containers sharing network and storage.
- **Deployment:** manages a set of identical Pod replicas and performs rolling updates.
- **Service:** gives a stable IP/DNS name and load balancing for a set of Pods selected by labels.
- **Namespace:** logical separation of resources inside a cluster (e.g. dev, test, prod).
- **ConfigMap:** stores non-sensitive configuration such as APP_ENV.
- **Secret:** stores sensitive data (passwords, tokens) base64-encoded; dummy credentials are used in the lab.
- **PersistentVolume (PV) / PersistentVolumeClaim (PVC):** storage that exists independently of Pod lifecycles; a Pod uses a PVC, which is bound to a PV.

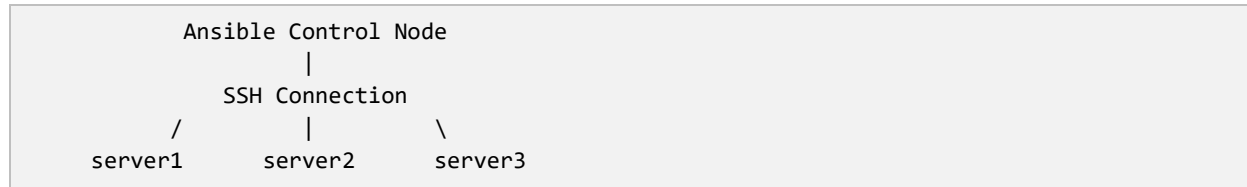
PersistentVolume	←bound→	PersistentVolumeClaim	←mounted by→	Pod
------------------	---------	-----------------------	--------------	-----

Kubernetes Service Types

- **ClusterIP:** default type; reachable only inside the cluster, used for internal communication.
- **NodePort:** exposes the service on a static port (30000–32767) of every node: <Node-IP>:<NodePort>.
- **LoadBalancer:** exposes the service through an external load balancer (cloud provider, or k3s ServiceLB locally).
- **ExternalName:** maps a service name to an external DNS name using a CNAME record.

Part B: Ansible – Configuration Management

Ansible is an agentless automation tool. A control node connects to managed nodes over SSH and executes tasks described in YAML playbooks. Modules such as apt, service and copy are idempotent: they only make a change when the system is not already in the desired state.



- **Inventory:** list of managed hosts and groups (inventory.ini).
- **Module:** unit of work, e.g. ping, apt, service, copy, shell.
- **Playbook:** YAML file with plays and tasks applied to a group of hosts.
- **become:** privilege escalation (sudo) for administrative tasks.

Case Study: Automated Installation and Configuration of Nginx

Problem: an organisation needs the same web-server environment on several Linux machines. Manual installation leads to configuration inconsistencies, human errors, longer deployment time and difficult maintenance. Solution: an Ansible playbook installs Nginx, starts it and deploys the same web page on all servers from one control node.

Code / Implementation

Part A – Step 1: Cluster

The same single-node k3s Kubernetes cluster as in Assignment 7 was used, with kubectl run from the toolbox container.

Step 2: Pod and Deployment

k8s/pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx-pod
spec:
  containers:
    - name: nginx
      image: nginx:1.27-alpine
      ports:
        - containerPort: 80
```

k8s/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
```

```

metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27-alpine
          ports:
            - containerPort: 80

kubectl apply -f k8s/pod.yaml
kubectl get pods
kubectl apply -f k8s/deployment.yaml
kubectl get deployments
kubectl expose deployment nginx-deployment --type=NodePort --port=80

```

Step 3: Namespace, ConfigMap and Secret

```

kubectl create namespace dev
kubectl get namespaces
kubectl create configmap app-config --from-literal=APP_ENV=production -n dev
kubectl create secret generic db-secret --from-literal=username=admin --from-
literal=password=admin123 -n dev # dummy lab credentials

```

k8s/config-pod.yaml – Pod that consumes the ConfigMap and Secret

```

# Pod that reads the ConfigMap and Secret as environment variables
apiVersion: v1
kind: Pod
metadata:
  name: config-demo
  namespace: dev
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "echo APP_ENV=$APP_ENV DB_USER=$DB_USER && sleep 3600"]
      env:
        - name: APP_ENV
          valueFrom:
            configMapKeyRef:
              name: app-config
              key: APP_ENV
        - name: DB_USER
          valueFrom:

```

```
secretKeyRef:
  name: db-secret
  key: username
```

Step 4: PersistentVolume and PersistentVolumeClaim

k8s/storage.yaml

```
# PersistentVolume -> PersistentVolumeClaim -> Pod
apiVersion: v1
kind: PersistentVolume
metadata:
  name: demo-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  storageClassName: manual
  hostPath:
    path: /tmp/demo-pv
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: demo-pvc
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: manual
  resources:
    requests:
      storage: 500Mi
---
apiVersion: v1
kind: Pod
metadata:
  name: storage-pod
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "echo 'Data stored on a PersistentVolume' > /data/message.txt
&& sleep 3600"]
      volumeMounts:
        - name: data
          mountPath: /data
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: demo-pvc
```

Step 5: Service types

k8s/services.yaml – ClusterIP, NodePort, LoadBalancer and ExternalName

```
# The four Kubernetes Service types, all pointing at the nginx Deployment
apiVersion: v1
kind: Service
metadata:
  name: nginx-clusterip
spec:
  type: ClusterIP
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: nginx-nodeport
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30081
---
apiVersion: v1
kind: Service
metadata:
  name: nginx-loadbalancer
spec:
  type: LoadBalancer
  selector:
    app: nginx
  ports:
    - port: 8088
      targetPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: external-service
spec:
  type: ExternalName
  externalName: example.com
```

Part B – Step 6: Ansible lab environment

Three Ubuntu 24.04 containers with SSH were used as managed web servers, and a separate container with ansible-core as the control node, all on one Docker network. The control node authenticates with an SSH key; the ansible user has passwordless sudo.

ansible/compose.yaml

```
# Lab environment: 1 Ansible control node + 3 Ubuntu web servers
services:
  control:
    build: { context: ., dockerfile: Dockerfile.control }
    container_name: ansible-control
    volumes:
      - ./:/ansible
    networks: [lab]
  server1:
    build: { context: ., dockerfile: Dockerfile.node }
    container_name: server1
    hostname: server1
    ports: ["127.0.0.1:8181:80"]
    networks: [lab]
  server2:
    build: { context: ., dockerfile: Dockerfile.node }
    container_name: server2
    hostname: server2
    networks: [lab]
  server3:
    build: { context: ., dockerfile: Dockerfile.node }
    container_name: server3
    hostname: server3
    networks: [lab]

networks:
  lab:
```

ansible/inventory.ini

```
[webservers]
server1
server2
server3

[webservers:vars]
ansible_user=ansible
ansible_ssh_private_key_file=/root/.ssh/ansible_lab_key
ansible_python_interpreter=/usr/bin/python3
```

ansible/install-nginx.yml – playbook

```
---
- name: Configure Web Servers
  hosts: webservers
```

```

become: yes

tasks:
  - name: Install Nginx
    apt:
      name: nginx
      state: present
      update_cache: yes

  - name: Start Nginx
    service:
      name: nginx
      state: started
      enabled: yes

  - name: Deploy Web Page
    copy:
      content: |
        <h1>Web Server Configured Using Ansible</h1>
        <p>Host: {{ inventory_hostname }}</p>
      dest: /var/www/html/index.html

```

Step 7: Run Ansible

```

ansible all -i inventory.ini -m ping
ansible-playbook -i inventory.ini install-nginx.yml
curl http://server1
ansible-playbook -i inventory.ini install-nginx.yml # second run - idempotency

```

Output

Part A: Kubernetes Objects and Services

1. Pod created and running:

```

toolbox (kubectl) — connected to k3s cluster
/work/a8 # kubectl apply -f k8s/pod.yaml
pod/nginx-pod created

/work/a8 # kubectl wait --for=condition=Ready pod/nginx-pod --timeout=120s
pod/nginx-pod condition met

/work/a8 # kubectl get pods -o wide

```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
nginx-pod	1/1	Running	0	3s	10.42.0.11	k3s-node	<none>	<none>

2. Deployment with 3 replicas, and exposed as a NodePort Service:

```
toolbox (kubectl) — connected to k3s cluster
/work/a8 # kubectl apply -f k8s/deployment.yaml
deployment.apps/nginx-deployment created

/work/a8 # kubectl rollout status deployment/nginx-deployment --timeout=120s
Waiting for deployment "nginx-deployment" rollout to finish: 0 of 3 updated replicas are available...
Waiting for deployment "nginx-deployment" rollout to finish: 1 of 3 updated replicas are available...
Waiting for deployment "nginx-deployment" rollout to finish: 2 of 3 updated replicas are available...
deployment "nginx-deployment" successfully rolled out

/work/a8 # kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment    3/3     3            3           3s

/work/a8 # kubectl get pods -l app=nginx
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-5b9fc7f7dd-c2qxz   1/1     Running   0           3s
nginx-deployment-5b9fc7f7dd-jrltt   1/1     Running   0           3s
nginx-deployment-5b9fc7f7dd-js8l4   1/1     Running   0           3s

/work/a8 # kubectl expose deployment nginx-deployment --type=NodePort --port=80
service/nginx-deployment exposed

/work/a8 # kubectl get service nginx-deployment
NAME                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
nginx-deployment    NodePort    10.43.157.184 <none>        80:30146/TCP     0s
```

3. Namespace, ConfigMap and Secret created, and consumed by a Pod as environment variables:

```
toolbox (kubectl) — connected to k3s cluster
/work/a8 # kubectl create namespace dev
namespace/dev created

/work/a8 # kubectl get namespaces
NAME          STATUS    AGE
default       Active   6m25s
dev           Active   0s
kube-node-lease Active   6m25s
kube-public   Active   6m25s
kube-system   Active   6m25s

/work/a8 # kubectl create configmap app-config --from-literal=APP_ENV=production -n dev
configmap/app-config created

/work/a8 # kubectl create secret generic db-secret --from-literal=username=admin --from-literal=password=admin123 -n dev
secret/db-secret created

/work/a8 # kubectl get configmap app-config -n dev -o jsonpath='{.data}'; echo
{"APP_ENV":"production"}

/work/a8 # kubectl get secret db-secret -n dev
NAME          TYPE      DATA   AGE
db-secret     Opaque    2       1s

/work/a8 # kubectl get secret db-secret -n dev -o jsonpath='{.data.username}' | base64 -d; echo
admin

/work/a8 # kubectl apply -f k8s/config-pod.yaml
pod/config-demo created

/work/a8 # kubectl wait --for=condition=Ready pod/config-demo -n dev --timeout=120s
pod/config-demo condition met

/work/a8 # kubectl logs config-demo -n dev
APP_ENV=production DB_USER=admin
```

4. PersistentVolume and PersistentVolumeClaim bound, and data written by the Pod to the volume:

```
toolbox (kubectl) — connected to k3s cluster
/work/a8 # kubectl apply -f k8s/storage.yaml
persistentvolume/demo-pv created
persistentvolumeclaim/demo-pvc created
pod/storage-pod created

/work/a8 # kubectl wait --for=condition=Ready pod/storage-pod --timeout=120s
pod/storage-pod condition met

/work/a8 # kubectl get pv
NAME          CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM          STORAGECLASS   VOLUMEATTRIBUTESCLASS   REASON   AGE
demo-pv       1Gi        RWO            Retain           Bound    default/demo-pvc manual         <unset>                  <unset>   6s

/work/a8 # kubectl get pvc
NAME          STATUS   VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
demo-pvc     Bound    demo-pv  1Gi        RWO            manual         <unset>                  6s

/work/a8 # kubectl exec storage-pod -- cat /data/message.txt
Data stored on a PersistentVolume
```

5. All four Service types created (the LoadBalancer received an external IP from k3s ServiceLB):

```
toolbox (kubectl) — connected to k3s cluster
/work/a8 # kubectl apply -f k8s/services.yaml
service/nginx-clusterip created
service/nginx-nodeport created
service/nginx-loadbalancer created
service/external-service created

/work/a8 # kubectl get services
NAME                TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
external-service    ExternalName  <none>           example.com      <none>           16s
kubernetes           ClusterIP     10.43.0.1        <none>           443/TCP          7m
nginx-clusterip      ClusterIP     10.43.249.196    <none>           80/TCP           16s
nginx-deployment     NodePort      10.43.157.184    <none>           80:30146/TCP     39s
nginx-loadbalancer   LoadBalancer 10.43.107.81     172.17.0.3      8088:31667/TCP   16s
nginx-nodeport       NodePort      10.43.67.126     <none>           80:30081/TCP     16s
```

6. Testing each Service type – ClusterIP from inside the cluster, NodePort, LoadBalancer, and ExternalName DNS resolving to example.com:

```
toolbox (kubectl) — connected to k3s cluster
/work/a8 # kubectl run test-client --image=busybox:1.36 --restart=Never --rm -i --quiet -- sh -c 'wget -qO- http://nginx-clusterip | grep -o "<title>.*</title>"'
<title>Welcome to nginx!</title>

/work/a8 # curl -s http://localhost:30081 | grep -o '<title>.*</title>'
<title>Welcome to nginx!</title>

/work/a8 # curl -s http://172.17.0.3:8088 | grep -o '<title>.*</title>'
<title>Welcome to nginx!</title>

/work/a8 # kubectl run dns-client --image=busybox:1.36 --restart=Never --rm -i --quiet -- sh -c 'sleep 2; nslookup external-service.default.svc.cluster.local | grep canonical | sort -u'
external-service.default.svc.cluster.local    canonical name = example.com
```



Part B: Ansible Case Study

7. Lab environment – control node and three web servers running:

Windows PowerShell

```
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment8_K8s_Ansible\ansible> docker compose up -d
Container server3 Started
Container server1 Started
Container ansible-control Started
Container server2 Started
```

```
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment8_K8s_Ansible\ansible> docker compose ps
```

NAME	IMAGE	STATUS	PORTS
ansible-control	ansible-control	Up 2 minutes	
server1	ansible-server1	Up 2 minutes	127.0.0.1:8181->80/tcp
server2	ansible-server2	Up 2 minutes	22/tcp, 80/tcp
server3	ansible-server3	Up 2 minutes	22/tcp, 80/tcp

8. Inventory and connectivity test – all servers reply pong:

```
ansible-control (Ansible control node)
root@ansible-control:/ansible# ansible --version | head -2
ansible [core 2.17.14]
  config file = /ansible/ansible.cfg

root@ansible-control:/ansible# cat inventory.ini
[webservers]
server1
server2
server3

[webservers:vars]
ansible_user=ansible
ansible_ssh_private_key_file=/root/.ssh/ansible_lab_key
ansible_python_interpreter=/usr/bin/python3

root@ansible-control:/ansible# ansible-inventory -i inventory.ini --graph
@all:
|--@ungrouped:
|--@webservers:
| |--server1
| |--server2
| |--server3

root@ansible-control:/ansible# ansible all -i inventory.ini -m ping
server1 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
server2 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
server3 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
```

9. Playbook run – Nginx installed, started and web page deployed on all three servers (changed=3):

```
ansible-control (Ansible control node)
root@ansible-control:/ansible# ansible-playbook -i inventory.ini install-nginx.yml

PLAY [Configure Web Servers] *****

TASK [Gathering Facts] *****
ok: [server2]
ok: [server3]
ok: [server1]

TASK [Install Nginx] *****
changed: [server3]
changed: [server2]
changed: [server1]

TASK [Start Nginx] *****
changed: [server2]
changed: [server3]
changed: [server1]

TASK [Deploy Web Page] *****
changed: [server3]
changed: [server1]
changed: [server2]

PLAY RECAP *****
server1          : ok=4    changed=3    unreachable=0    failed=0    skipped=0    rescued=0
  ignored=0
server2          : ok=4    changed=3    unreachable=0    failed=0    skipped=0    rescued=0
  ignored=0
server3          : ok=4    changed=3    unreachable=0    failed=0    skipped=0    rescued=0
  ignored=0
```

10. Verification – every server returns the page, and Nginx is running:

```
ansible-control (Ansible control node)
root@ansible-control:/ansible# curl -s http://server1
<h1>Web Server Configured Using Ansible</h1>
<p>Host: server1</p>

root@ansible-control:/ansible# curl -s http://server2
<h1>Web Server Configured Using Ansible</h1>
<p>Host: server2</p>

root@ansible-control:/ansible# curl -s http://server3
<h1>Web Server Configured Using Ansible</h1>
<p>Host: server3</p>

root@ansible-control:/ansible# ansible webservers -i inventory.ini -b -m shell -a 'nginx -v 2>&1; service nginx status'
server3 | CHANGED | rc=0 >>
nginx version: nginx/1.24.0 (Ubuntu)
* nginx is running
server2 | CHANGED | rc=0 >>
nginx version: nginx/1.24.0 (Ubuntu)
* nginx is running
server1 | CHANGED | rc=0 >>
nginx version: nginx/1.24.0 (Ubuntu)
* nginx is running
```



11. Second playbook run – changed=0 on every server, demonstrating idempotent configuration:

```
ansible-control (Ansible control node)
root@ansible-control:/ansible# ansible-playbook -i inventory.ini install-nginx.yml

PLAY [Configure Web Servers] *****

TASK [Gathering Facts] *****
ok: [server1]
ok: [server2]
ok: [server3]

TASK [Install Nginx] *****
ok: [server2]
ok: [server1]
ok: [server3]

TASK [Start Nginx] *****
ok: [server3]
ok: [server2]
ok: [server1]

TASK [Deploy Web Page] *****
ok: [server2]
ok: [server1]
ok: [server3]

PLAY RECAP *****
server1      : ok=4    changed=0    unreachable=0    failed=0    skipped=0    rescued=0
  ignored=0
server2      : ok=4    changed=0    unreachable=0    failed=0    skipped=0    rescued=0
  ignored=0
server3      : ok=4    changed=0    unreachable=0    failed=0    skipped=0    rescued=0
  ignored=0
```

Combined DevOps Workflow

Developer → GitHub → Jenkins → Build / Test → Containerized Application (Docker)
→ Kubernetes → Helm Deployment → Application Services → Ansible Infrastructure Configuration

Conclusion

In this assignment, I learned about the main Kubernetes objects – Pod, Deployment, Service, Namespace, ConfigMap, Secret, PersistentVolume and PersistentVolumeClaim – and the differences between ClusterIP, NodePort, LoadBalancer and ExternalName services, all of which I created and tested on a real cluster. In the Ansible case study I automated the installation and configuration of Nginx on three servers from a single control node using an inventory and a playbook; the second run changed nothing, which demonstrated idempotent, repeatable, agentless configuration management.

References

1. Kubernetes Workloads – <https://kubernetes.io/docs/concepts/workloads/>

2. Kubernetes Services – <https://kubernetes.io/docs/concepts/services-networking/service/>
3. Kubernetes Persistent Volumes – <https://kubernetes.io/docs/concepts/storage/persistent-volumes/>
4. ConfigMaps and Secrets – <https://kubernetes.io/docs/concepts/configuration/>
5. Ansible Documentation – <https://docs.ansible.com/>
6. Ansible Playbook Guide – https://docs.ansible.com/ansible/latest/playbook_guide/

Submission Links

GitHub Repository Link:

https://github.com/tanushkapatil/DevOps/tree/main/Assignment8_K8s_Ansible

Hosted / Output Link: <http://127.0.0.1:8181> (local – lab removed after testing)